VISION

AI 기반 웹 보안 시스템

목차

01 프로젝트 배경

02 프로젝트 개요

03 시스템 아키텍처

04 탐지 엔진

05 FastApi

06 관리자 페이지 구현

07 시연 결과

08 기대 효과

01 프로젝트 배경

유해 사이트 접속 위협 증가

사용자의 웹사이트 접속 과정에서
유해 사이트에 접근할 가능성이 증가하고있다

웹 접속 관리 시스템의 필요성

웹사이트 접속을 허용·차단 기준에 따라
체계적으로 관리할 수 있는 시스템이 필요하다

정책 기준 관리 기능 필요성

빠르게 변화하는 유해 사이트를 효율적으로
관리하기위한 시스템이 필요하다

02 프로젝트 개요

사용자의 웹사이트 접속 요청을 분석하고,
정책에 따라 허용·차단을 관리하는 웹 접속 보안 관리 시스템

탐지 엔진

- 접속 요청 확인
- 허용 / 차단 처리
- 처리 결과 전달

관리자 페이지

- 정책 설정 및 관리
- 로그 및 분석 결과 조회
- 운영 현황 모니터링

FastAPI

- URL 및 도메인 분석
- 분석 결과 전달

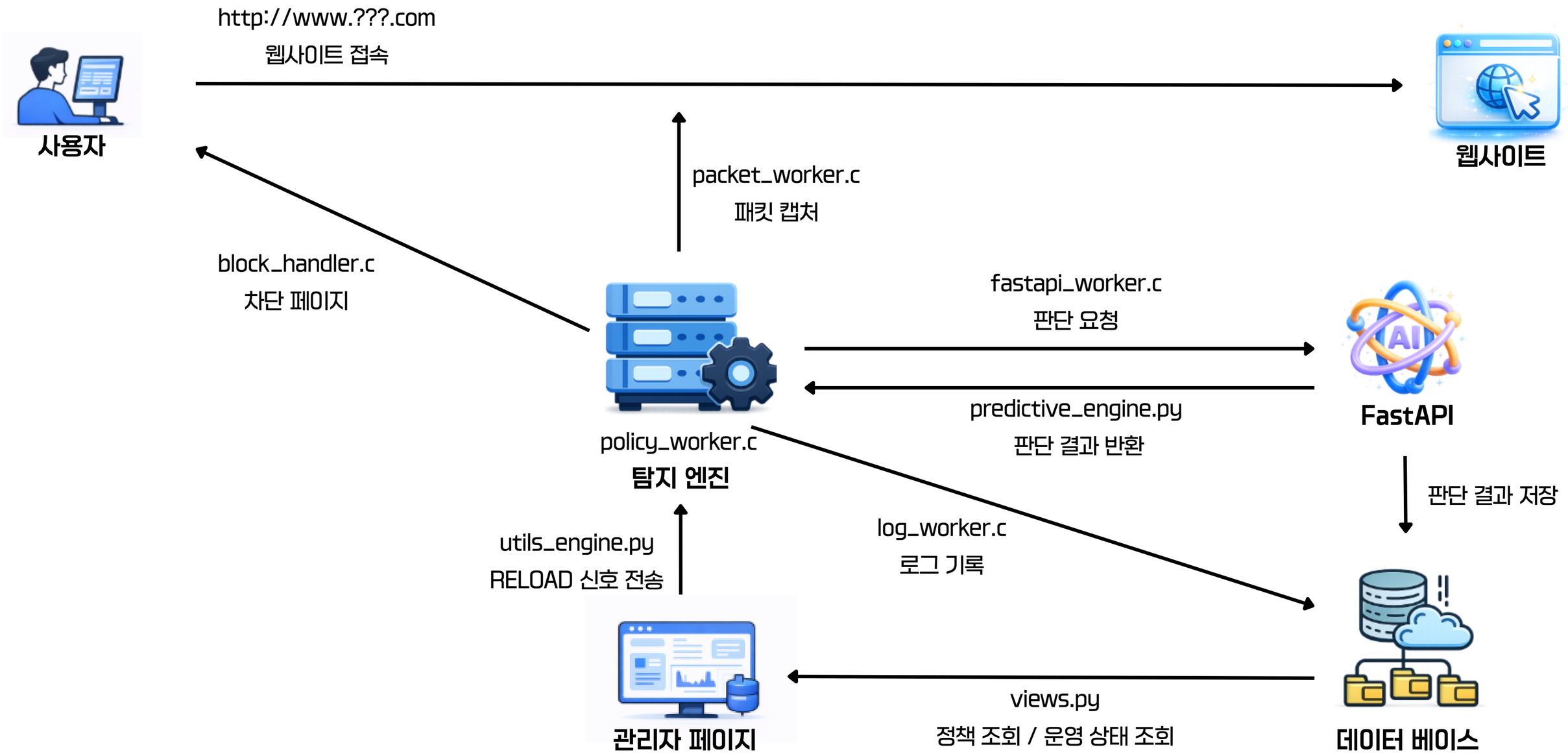
DataBase

- 로그 저장 및 관리
- 정책 데이터 저장

Technology Stack

Python · Django · FastAPI · MySQL · JavaScript · Bootstrap · C

03 시스템 아키텍처


http://www.???.com
웹사이트 접속
사용자
웹사이트
packet_worker.c
패킷 캡처
block_handler.c
차단 페이지
fastapi_worker.c
판단 요청
predictive_engine.py
판단 결과 반환
FastAPI
policy_worker.c
탐지 엔진
판단 결과 저장
log_worker.c
로그 기록
utils_engine.py
RELOAD 신호 전송
관리자 페이지
views.py
정책 조회 / 운영 상태 조회
데이터 베이스

04 탐지엔진 : 핵심 파일

packet_worker.c

역할 : 이 파일은 네트워크 패킷을 캡처해서
HTTP 요청을 분석하고 차단 여부를
결정하는 핵심 모듈

fastapi_worker.c

역할 : 이 파일은 차단되지 않은 URL 을
FastAPI 앱에 전달하고,
학습된 AI 모델을 통해 해당 URL의
유해 확률을 받아오는 모듈

policy_worker.c

역할 : 이 파일은 관리자 페이지에서
변경된 정책을 엔진에 적용하고,
FastAPI 재검토 요청을 실행하는 모듈

04 탐지엔진 : 핵심 기술

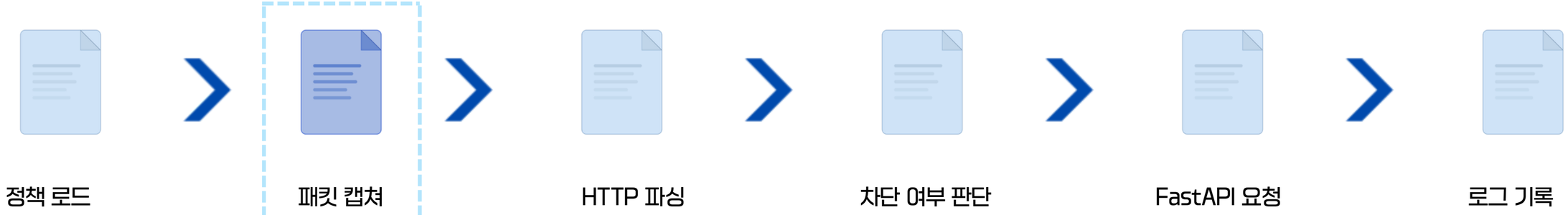
```
void policy_load_or_reload( void );

int build_domain_from_db( MYSQL* conn,  DomainPolicy***
                                out_table,  size_t* out_bucket );

int build_regex_from_db( MYSQL* conn, RegexPolicy**
                                out_arr, int* out_count );
```

- 데이터 베이스를 통해 현재 적용중인 정책을 로드해 온다.
- 도메인 정책은 전역 해쉬 테이블로 생성 한다.
- 정규표현식 정책은 전역 배열로 생성 한다.

04 탐지엔진 : 핵심 기술

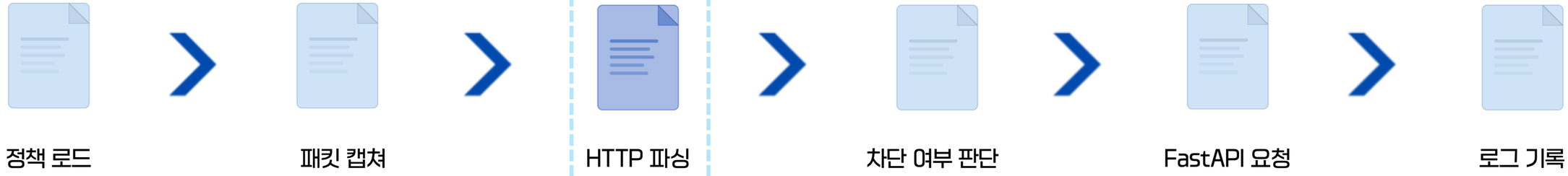



```
pcap_compile( handle, &fp, "tcp dst port 80", 0,
                    PCAP_NETMASK_UNKNOWN );
pcap_setfilter( handle,  &fp );

pcap_loop( handle,  0,  packet_handler,  NULL );
```

- TCP의 목적지 포트 번호가 80 (HTTP) 인 패킷만 캡쳐 하도록 설정한다.
- 적용된 규칙으로 패킷을 무한히 받도록 설정 한다.

04 탐지엔진 : 핵심 기술

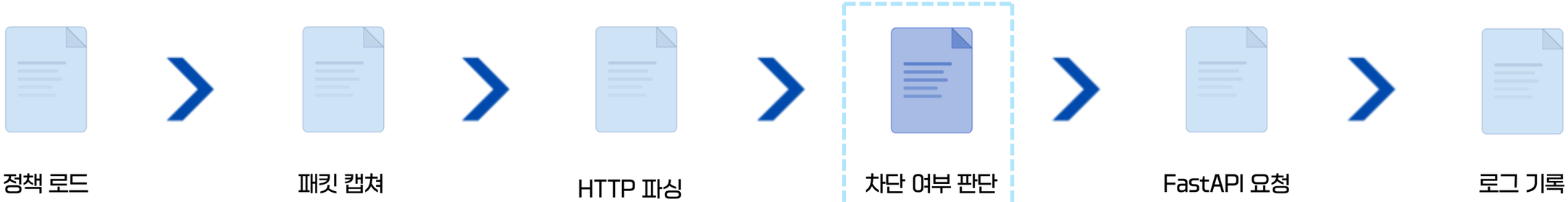



```
http_parse_request( const uint8_t* payload,  size_t len,
  uint32_t src_ip,  uint16_t dst_port, ParserPacket* parsed_pkt );
```

```
typedef struct {
    uint32_t src_ip;
    uint16_t dst_port;
    char method[MAX_METHOD_LEN];
    char domain[MAX_DOMAIN_LEN]
    char url[MAX_URL_LEN];
    char query[MAX_QUERY_LEN];
} ParserPacket;
```

- HTTP 패킷을 필요한 정보를 구조체에 저장한다.
- 클라이언트의 IP주소, 목적지 포트 번호, 요청 메서드, URL, 쿼리스트링을 파싱한다.

04 탐지엔진 : 핵심 기술



- 도메인 정책과 일치 하는지 해시 값 과 문자열을 비교 한 후 일치 하면 정책 이름을 기록 하고 차단한다.

```
while (iter) {
  if ( iter->hash_cache == hash && strcmp(iter->domain, domain) == 0) {
    result = iter->policy_name;
    strncpy(content, iter-<domain, MAX_DOMAIN_LEN - 1 );
    content[MAX_DOMAIN_LEN - 1] = '₩0';
    break; }
  iter = iter->p_next;
}
```

- 정규 표현식 정책과 일치 하는지 배열을 순회하며 비교 한 후 일치 하면 정책 이름을 기록 하고 차단한다.

```
for (int i = 0; i < regex_arr_count; i++) {
    if (regexec(&regex_arr_ptr[i].compiled, url, 0, NULL, 0) == 0) {
      result = regex_arr_ptr[i].policy_name;
      strncpy(content,regex_arr_ptr[i].pattern ,MAX_REGEX_LEN - 1);
      content[MAX_REGEX_LEN - 1] = '₩0';
      break; }
}
```

04 탐지엔진 : 핵심 기술

- FastAPI 에 요청을 보내기 위해 curl을 설정한다.
- FastAPI 에 GET요청을 할 URL을 준비한다.
- FastAPI 가 돌려준 JSON을 파싱하여 유해 확률을 기록한다.

```
curl_easy_setopt( curl, CURLOPT_URL, api_url );
curl_easy_setopt( curl, CURLOPT_WRITEFUNCTION, WriteCallback);
curl_easy_setopt( curl, CURLOPT_WRITEDATA, (void *)&chunk);
```

```
snprintf( api_url, sizeof(api_url), "http://192.168.100.25:8000/predict?url=%s", url );
```

```
char *pos = strstr(chunk.memory, "₩"collect_probability₩"");
if(pos) {sscanf(pos, "₩"collect_probability₩" : %f", &probability); }
```

04 탐지엔진 : 핵심 기술

- 데이터 베이스에 기록하기 위해 stmt 객체를 준비한다.
- 차단 기록을 HTTP 파싱된 정보와 함께 기록한다.
- 차단 되지 않은 기록을 FastAPI 요청 기록과 함께 기록한다.

```
mysql_stmt_prepare(stmt, sql, (unsigned long)strlen(sql));

insert_integrated(integrated_stmt, &block_arr[idx]);

insert_ai(ai_stmt, &req_arr[idx], &res_arr[idx]);
```

5-1 AI 학습

수치형 데이터

1. 전체 URL의 길이
2. 도메인의 길이
3. 숫자 개수
4. 하이픈(-) 개수
5. 점(.) 개수
6. 문자열 복잡도
7. 도메인 확장자 개수

범주형 데이터

1. 도메인이 ip형식 인지의 여부
2. 도메인 확장자

1. 피쳐 추출

```
def insert_url_csv(url : str):
    url_len = len(url) # 전체 URL의 길이
    domain = get_domain(url) # 도메인
    domain_len = len(domain) # 도메인의 길이
    cnt_digit = get_cnt_digit(domain) # 숫자 개수
    cnt_hyphen = domain.count('_') # 하이픈의 개수
    cnt_dot = domain.count('.') # 점의 개수
    is_ip = get_is_ip(domain) # IP 여부
    entropy = get_entropy(domain) # 문자열 복잡도
    # 도메인 확장자의 개수
    cnt_domain_ext = get_cnt_domain_ext(domain)
    # 도메인 확장자
    domain_ext = get_domain_ext(domain)
```

2. 피쳐를 CSV파일에 추가

```
if not ((df == new_row_df.iloc[0]).all(axis=1)).any():
    if not ((df == new_row_df.iloc[0]).all(axis=1)).any():
        df = pd.concat([df,new_row_df], ignore_index=True)
        print("행이 추가되었습니다.")
    else:
        print("이미 동일한 행이 존재합니다. 추가하지 않습니다.")
        return

    df.to_csv('preprocessed.csv', index=False, encoding='utf-8')
```

5-1 AI : 머신러닝 모델 생성과정

1. 데이터 로드

```
def load_data(path) :
    df = pd.read_csv(path)
    return df
```

2.데이터 분리

```
def split_data(df:pd.DataFrame):
    X =  df.drop(columns=['domain'])
    X =  df.drop(columns=['is_harmful'])
    y = df['is_harmful']

    X_train, X_test, y_train,y_test = train_test_split(
        X,
        y,
        test_size=0.4,
        stratify = y
    )
    return X_train, X_test, y_train,y_test
```

3.파이플라인 생성

```
def build_pipeline() :
    categorical_features = ['is_ip', 'domain_ext']
    numeric_features = ['url_length','domain_length' ,
                        'count_digit', 'count_hyphen',
                        'count_dot', 'entropy',
                        'count_domain_extention']
    ...
    ...
    return pipeline
```

4.머신러닝 모델 훈련

```
def train_model(pipeline, X_train, y_train):
    pipeline.fit(X_train, y_train)
    return pipeline
```

5.머신러닝 모델 훈련 평가

```
def evaluate_model(pipeline, X_test, y_test):
    y_pred = pipeline.predict(X_test)
    print("Accuracy :", accuracy_score(y_test, y_pred))
    print("Classification Report")
    print(classification_report(y_test, y_pred))
```

6.머신러닝 모델 저장

```
def save_model(model, model_path:str):
    os.makedirs(os.path.dirname(model_path),
                exist_ok = True)
    joblib.dump(model, model_path)
    print(f"[OK] Saved model -> {model_path}")
```

데이터 분리

데이터 학습에 사용하지 않는 칼럼을 제거 후 모델 학습용 데이터를 60%, 테스트용 데이터를 40%로 분리

파이프라인 생성

범주형 데이터와 수치형 데이터를 구분한 뒤 훈련과 평가를 위한 파이프라인을 생성

머신러닝 모델 훈련 평가

학습된 모델(pipeline)을 사용하여 테스트 데이터의 예측 결과를 계산하고 모델 성능을 평가 후 정확도를 출력

머신러닝 모델 저장

학습시킨 머신러닝 모델을 FastAPI에서 사용하기 위해 저장

5-1 FastAPI : URL의 유해 확률

역할

- URL을 전달받아 유해확률 분석
- 분석결과를 JSON 형태로 반환
- 시스템 상태(CPU / MEMORY)를 제공

사용된 모듈 : libcurl FastAPI pandas joblib

시스템 흐름

C client

URL 수신 / libcurl 요청 전송

```
snprintf(api_url, sizeof(api_url),
         "http://192.168.000.000:8000/predict?url=%s", url);

  // curl핸들러 세팅
curl_easy_setopt(curl, CURLOPT_URL, api_url);
curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, WriteCallback);
curl_easy_setopt(curl, CURLOPT_WRITEDATA, (void *)&chunk);

// 결과
res = curl_easy_perform(curl);
```

FastAPI Server

특징 추출 이후 머신러닝 모델을 통해 유해 확률 반환

```
url_len = len(url)      // URL길이
domain = get_domain(url)     // 도메인 주소
domain_len = len(domain)     // 도메인 주소 길이
cnt_digit = get_cnt_digit(domain)     // 숫자 개수
cnt_hyphen = domain.count('-')     // "-" 의 개수
...
...
return {
    "pred" : pred,
    "collect_probability" : collect_probability[0][1] * 100
}
```

5-2 FastAPI : 실시간 성능

역할

- 서버의 CPU 사용률과 메모리 사용률을 실시간으로 확인하는 API
- 대시보드 또는 관리자 페이지에서 실시간 상태 확인
- 시스템 과부하 감지

사용된 모듈 : FastAPI psutil

시스템 흐름

1. 클라이언트가 runtime 으로 GET 요청 전송
2. FastAPI가 psutil을 이용하여 CPU 사용률과 메모리 사용률을 조회
3. 조회된 값을 소수점 1자리로 반올림
4. JSON 형태로 클라이언트에게 시스템 상태 반환

FastAPI

```
@app.get("/runtime")
def runtime():
    cpu = psutil.cpu_percent(interval=0.3)
    mem = psutil.virtual_memory().percent
    return {"cpu": round(cpu, 1), "memory": round(mem, 1)}
```

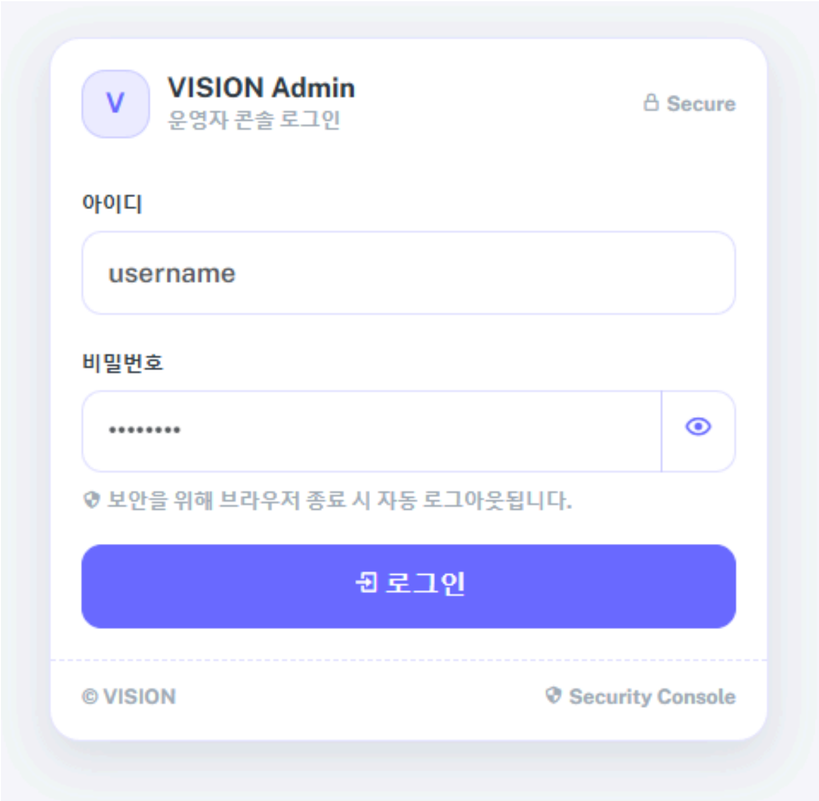
→

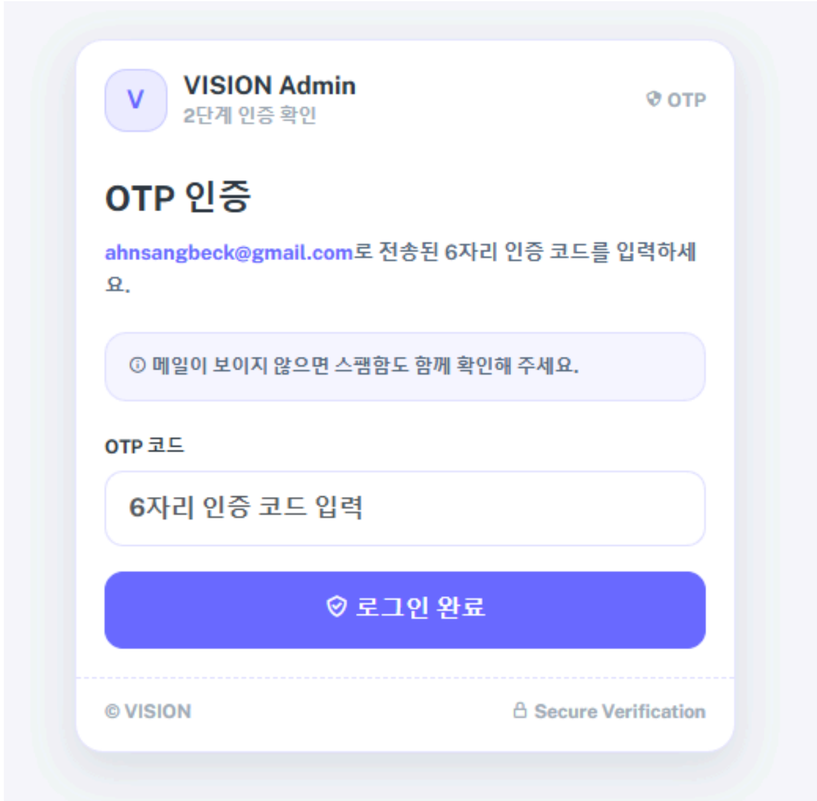
jason response

"cpu" : 23.4

"memory" : 51.6

06 관리자 페이지(로그인)

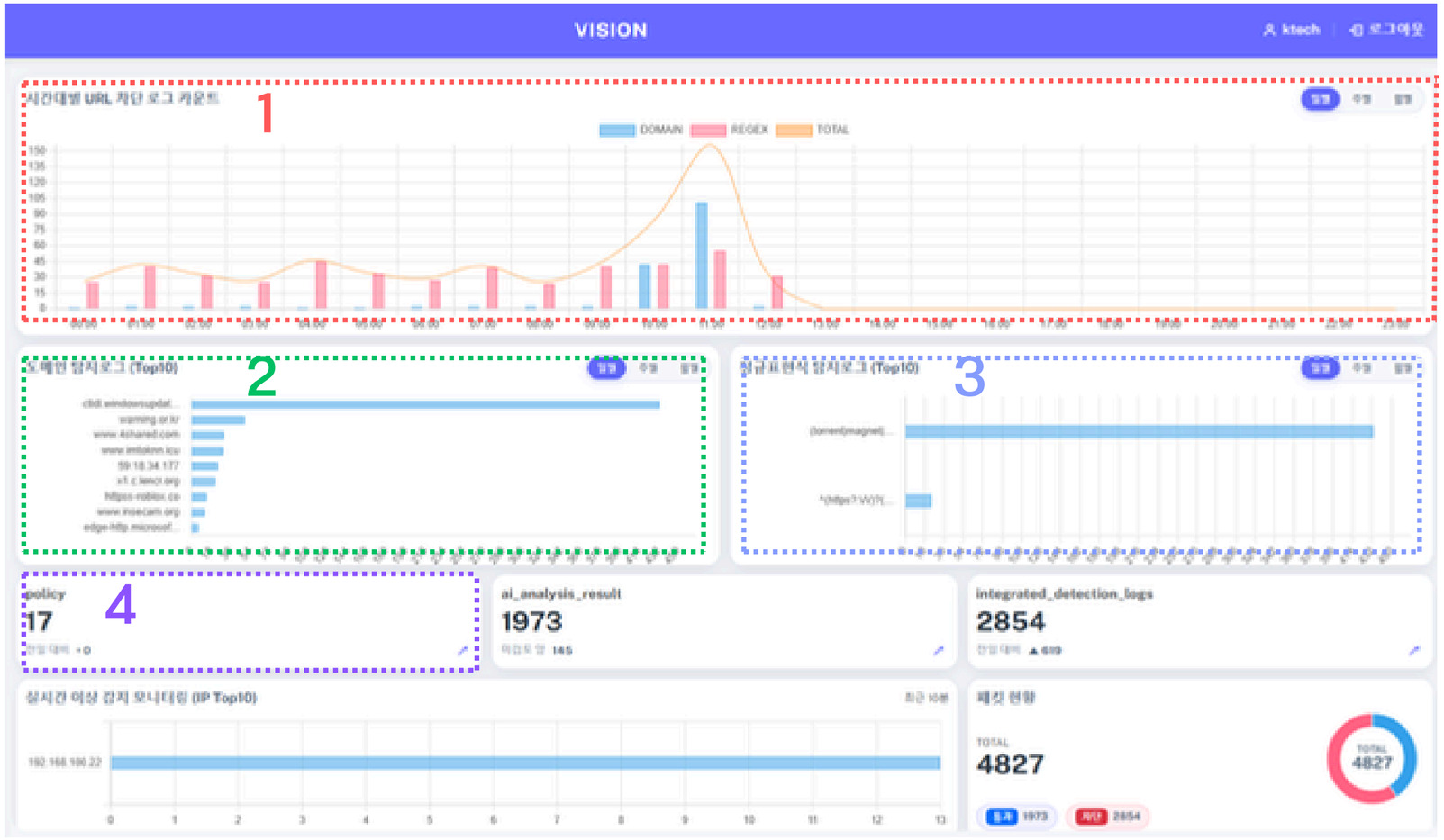




- 기본 로그인 : 관리자 계정의 아이디와 비밀번호를 입력하여 1차 인증을 수행합니다.
- OTP 인증 : 등록된 이메일 기반 인증코드를 입력해 2차 인증을 진행합니다.
- 접근 통제 강화 : 다단계 인증을 통해 비인가 사용자의 관리자 페이지 접근을 방지합니다.

6 관리자 페이지

대시보드



1. 시간대별 URL차단 로그 카운트

시간대별로 URL 차단 로그 수를 집계한 그래프로, 도메인 기반 차단과 정규표현식 기반 차단을 구분해 보여줍니다.

2. 도메인 탐지로그 (TOP 10)

차단된 도메인 중 상위 10개를 차단 횟수 기준으로 보여주는 그래프입니다.

3. 정규표현식 탐지로그(TOP 10)

차단에 사용된 정규표현식 정책 중 상위 10개를 차단 횟수 기준으로 보여주는 그래프입니다.

4. policy

현재 적용된 정책의 총 개수를 표시하며, 전날 대비 증가 및 감소한 수를 함께 제공합니다.

6 관리자 페이지

대시보드

5. ai_analysis_result

도메인 정책에 해당되지 않아 AI가 주가로 분석한 URL의 전체개수

6. intergrated_detection_log

차단된 http 요청에 대한 기록의 총 개수

7. 실시간 이상 감지 모니터링

차단된 http 요청 수를 기준으로 집계한 상위 10개의 호스트별 ip

8. 패킷 현황

서버에서 감지된 http 패킷의 총 개수

6-1 관리자 페이지 (정책 운영)

정책 현황

등록된 정책을 조회하고, 조건에 따라 검색/확인할 수 있는 페이지

(정책 선택)

(상세정보 확인)

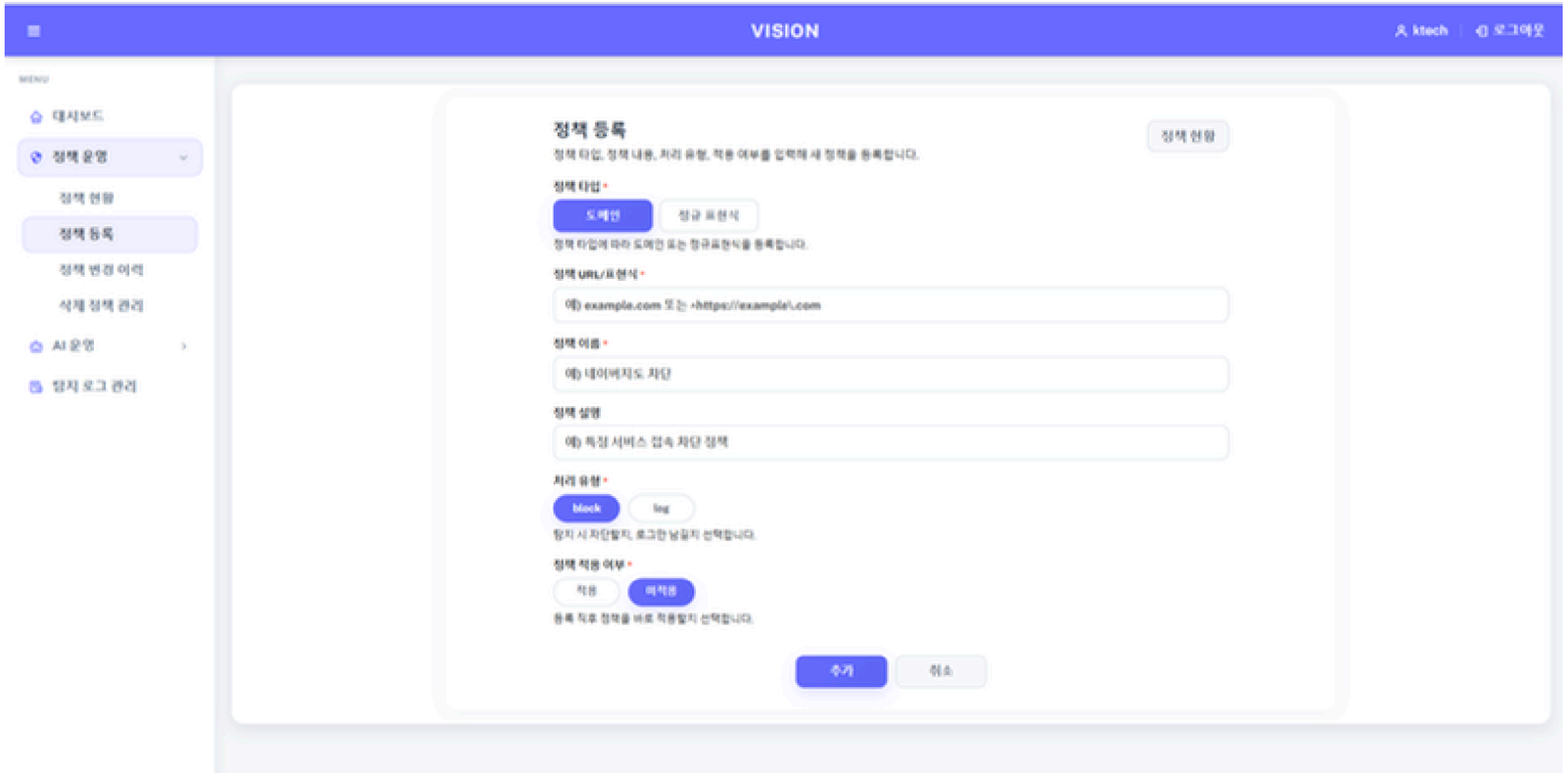
(설정 변경)

- 목록에서 정책을 선택하면 상세 정보를 확인할 수 있다.
- 상세 화면에서 적용 여부 및 처리 유형을 빠르게 수정할 수 있다.
- 상세 화면 우측상단에 수정 버튼을 통해 정책의 모든 정보를 수정할 수 있다.

6-2 관리자 페이지(정책 운영)

정책 등록

새로운 정책을 추가할 수 있는 페이지

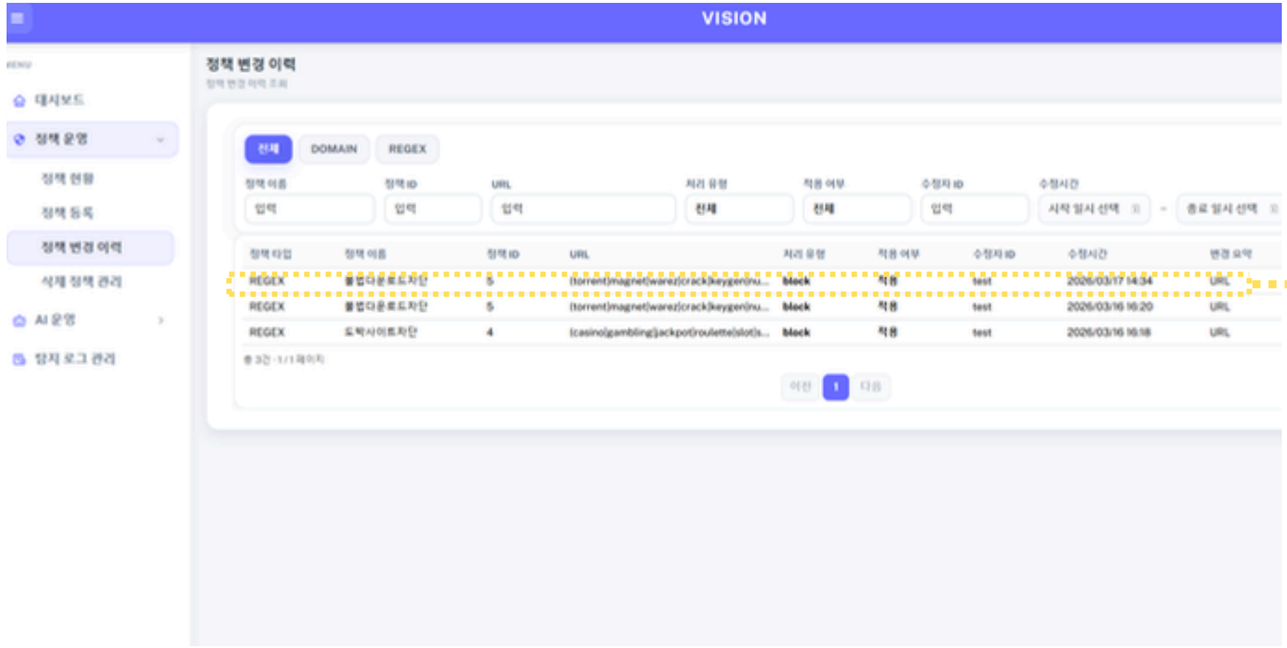


- 필수 입력 항목 (*) 미입력 시 오류 메시지 표시
- 이미 등록된 정책일 경우 중복 정책 안내
- 정책 유형 및 URL 입력을 통해 새로운 정책 등록 가능

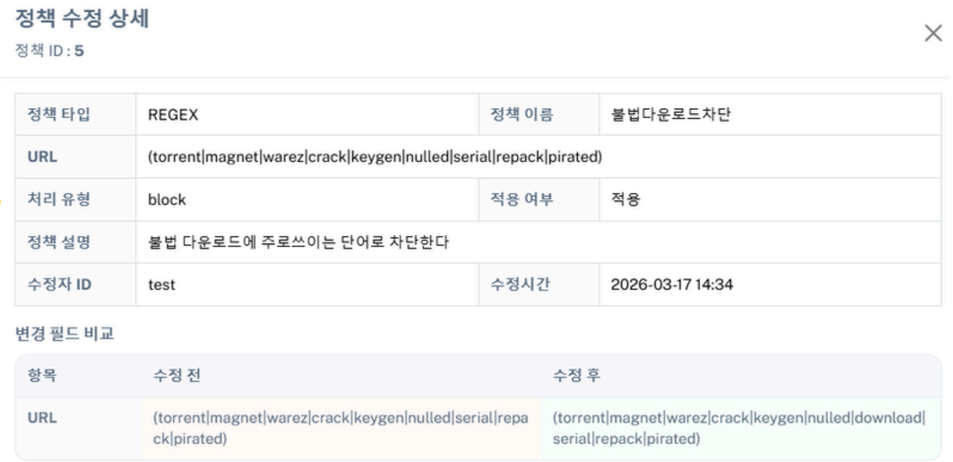
6-3 관리자 페이지(정책 운영)

정책 변경 이력

정책을 수정한 이력을 확인하는 페이지



(수정 내역 선택)



정책 수정 상세

정책 ID : 5

정책 타입	REGEX	정책 이름	불법다운로드차단
URL	(torrent|magnet|warez|crack|keygen|nulled|serial|repack|pirated)		
처리 유형	block	적용 여부	적용
정책 설명	불법 다운로드에 주로쓰이는 단어로 차단한다		
수정자 ID	test	수정시간	2026-03-17 14:34

변경 필드 비교

항목	수정 전	수정 후
URL	(torrent|magnet|warez|crack|keygen|nulled|serial|repack|pirated)	(torrent|magnet|warez|crack|keygen|nulled|download|serial|repack|pirated)

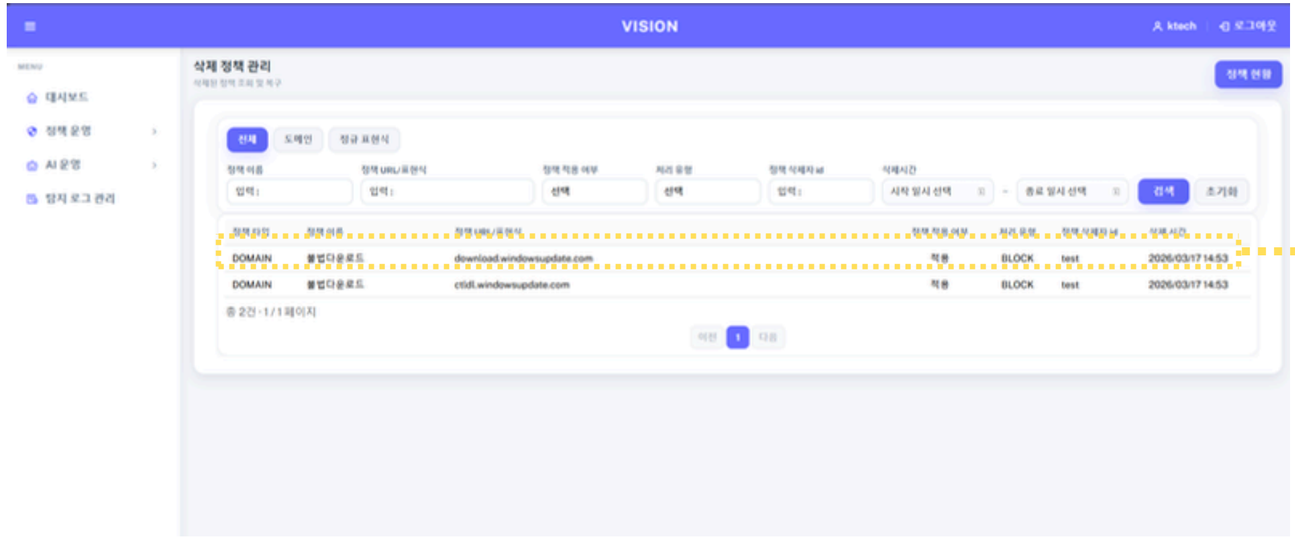
(수정 내역 상세 정보)

- 수정 이력 선택 시 상세 정보 모달 제공
- 변경 항목 및 수정 전 / 후 내용 비교 가능
- 정책 ID 기준으로 해당 정책의 전체 변경 이력 조회 가능

6-4 관리자 페이지(정책 운영)

삭제 정책 관리

삭제된 정책을 조회하고, 상세 정보 확인 후 복구할 수 있는 관리 페이지

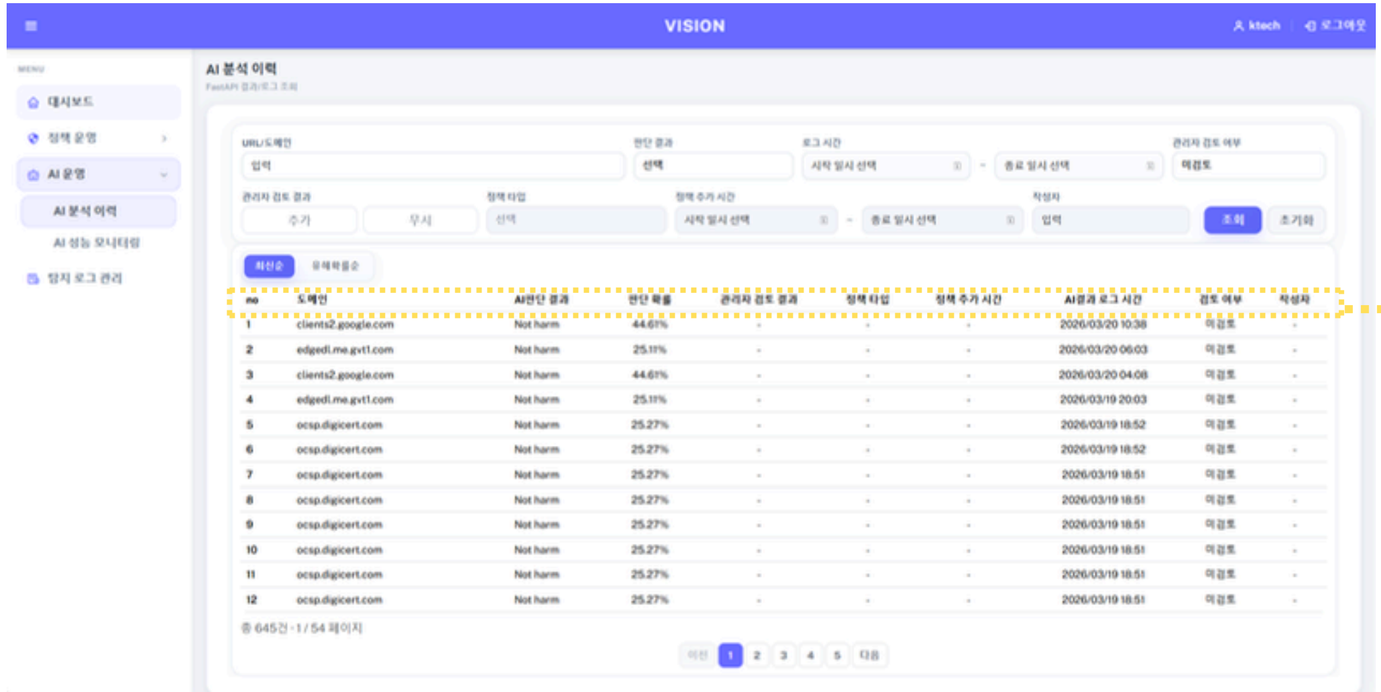


- 삭제 된 정책 선택시 상세 정보를 모달창으로 확인할 수 있습니다.
- 복구 버튼을 통해 삭제된 정책을 즉시 복구할 수 있습니다.
- 삭제 이력을 기반으로 정책 복구 및 운영 추적이 가능합니다.

6-6 관리자 페이지(AI 운영)

AI 분석 이력

AI 분석 결과를 조회하고 관리자 검토 및 정책 반영을 지원하는 페이지

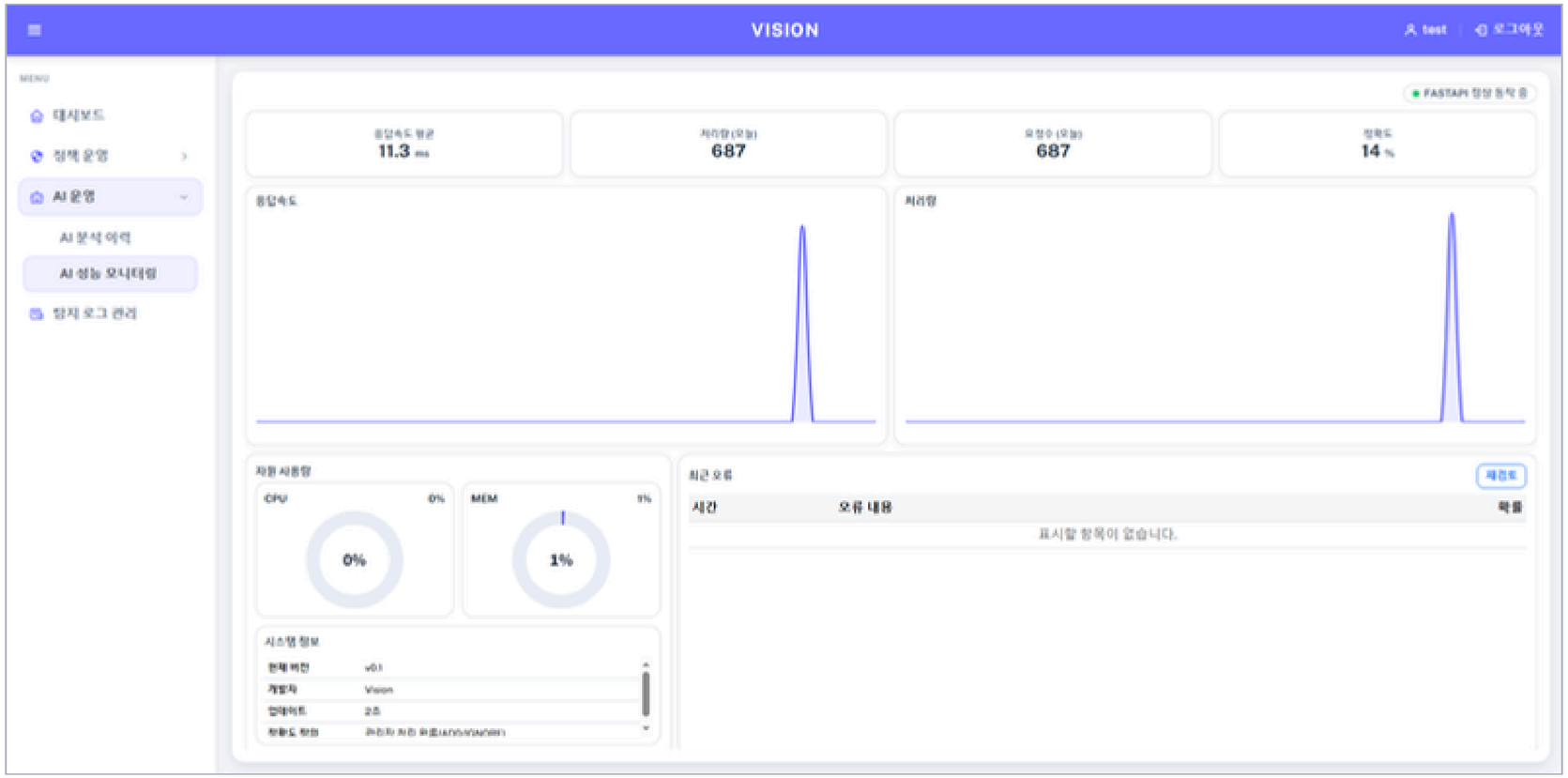


- AI 분석 이력 목록을 통해 판단 결과를 한눈에 확인할 수 있습니다.
- 상세 패널에서 URL, 도메인, AI 판단 결과, 확률, 중복 건수 등 세부 정보를 제공합니다.
- 관리자 검토 후 정책 추가 기능으로 정책 관리 페이지와 연계할 수 있습니다.
- 무시 기능을 통해 불필요한 대응을 제외하여 운영 효율성을 높일 수 있습니다.

6-7 관리자 페이지(AI 운영)

AI 성능 모니터링

AI 탐지 엔진의 처리 현황, 서버 상태, 오류 내역을 통합 모니터링 할 수 있는 페이지



1. 핵심 성능 지표를 상단 카드로 제공하여 현재 AI 처리 상태를 빠르게 확인할 수 있다.

2. 시간대별 그래프로 처리 흐름과 이상 징후를 시각적으로 분석할 수 있다.

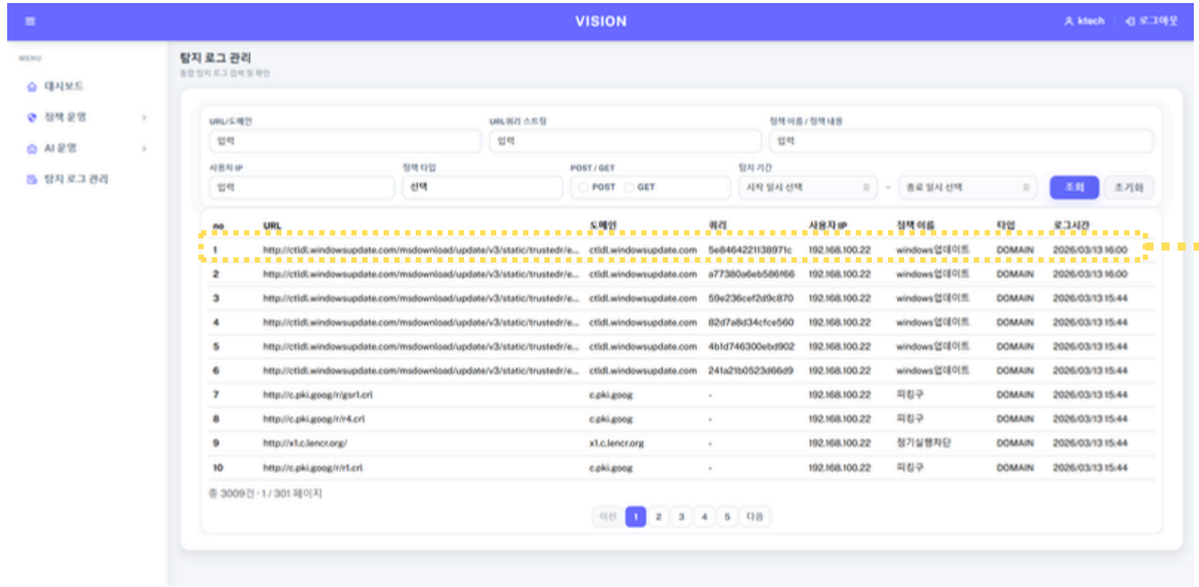
3. CPU / 메모리 사용량과 서버 정보를 통해 사용량을 빠르게 파악할 수 있다.

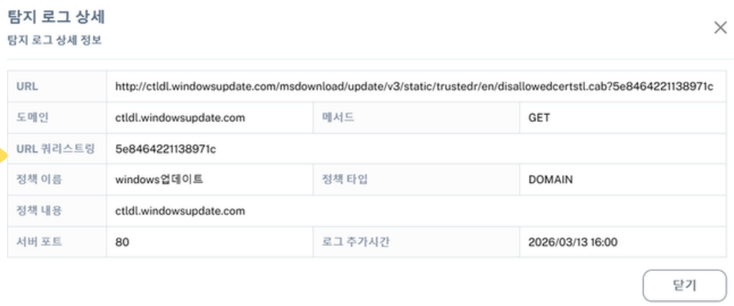
4. AI 서버 장애 발생 시 오류 내역에 관련 정보가 표시되며, 정상 작동 시에는 재검토 버튼을 통해 재검토를 수행할 수 있습니다.

6-8 관리자 페이지(탐지 로그 관리)

탐지 로그 관리

차단된 탐지 로그를 조회하고 상세 정보를 확인할 수 있는 페이지





- 차단된 URL 및 탐지 로그를 목록 형태로 조회할 수 있습니다.
- 선택한 로그의 URL, IP, 요청 방식, 정책 정보 등 상세 내용을 확인할 수 있습니다.
- 탐지 결과를 기반으로 차단 내역을 추적하고 운영 현황을 파악할 수 있습니다.